

Assignment 2:
Software Design Specification
Clothing Store Point-of-Sale System

Group 17:
Andres Ortiz
Max Winchester
Krish Patel

Overview

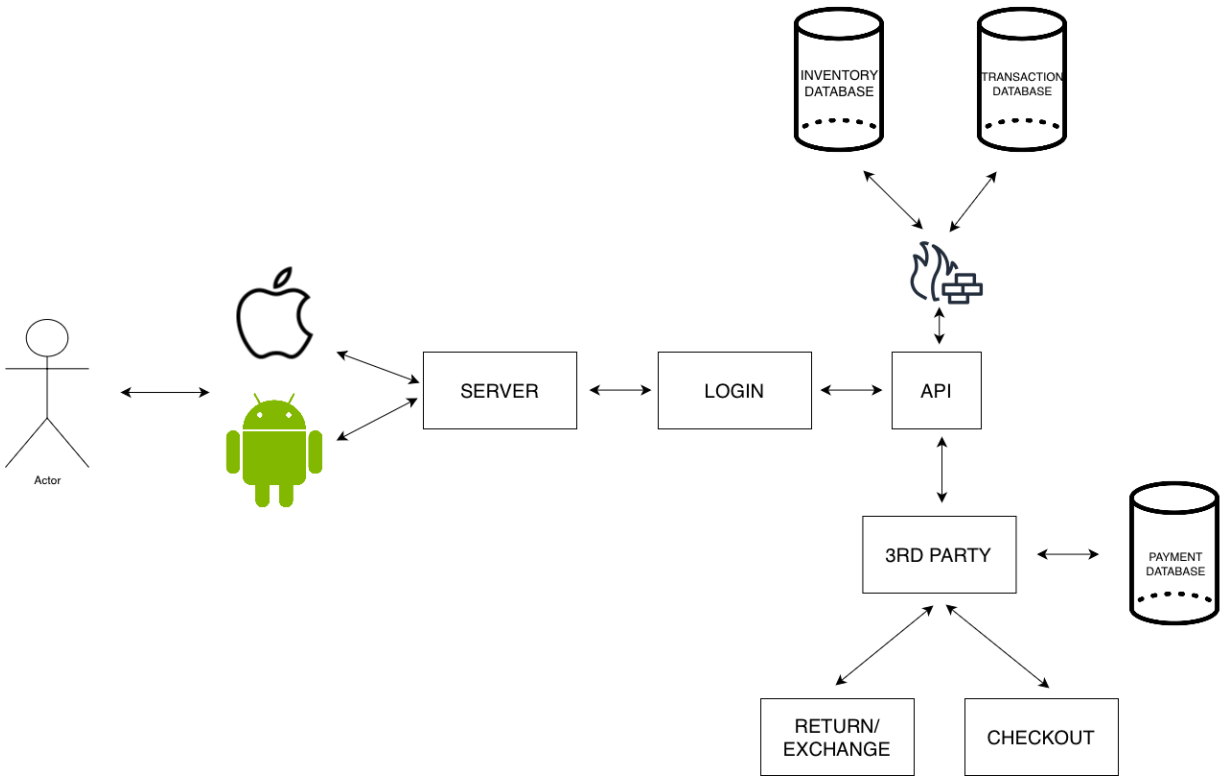
The point-of-sale system is designed not only to assist employees with sales and refunds but also to allow them to manage and update the store's inventory. Employees will have access to the system as well as the inventory database. Administrators will also be able to access a separate, secure transaction database. Both the inventory and the transaction databases will be backed up through a cloud and synchronized with the separate store locations.

The system will utilize external systems to process credit card and debit card transactions. A built-in barcode scanner and sorting system will allow employees to search for items in the database using unique ID codes for each item. Items are also traceable through specific attributes such as color, size, price, etc. The inventory is automatically updated after any transactions.

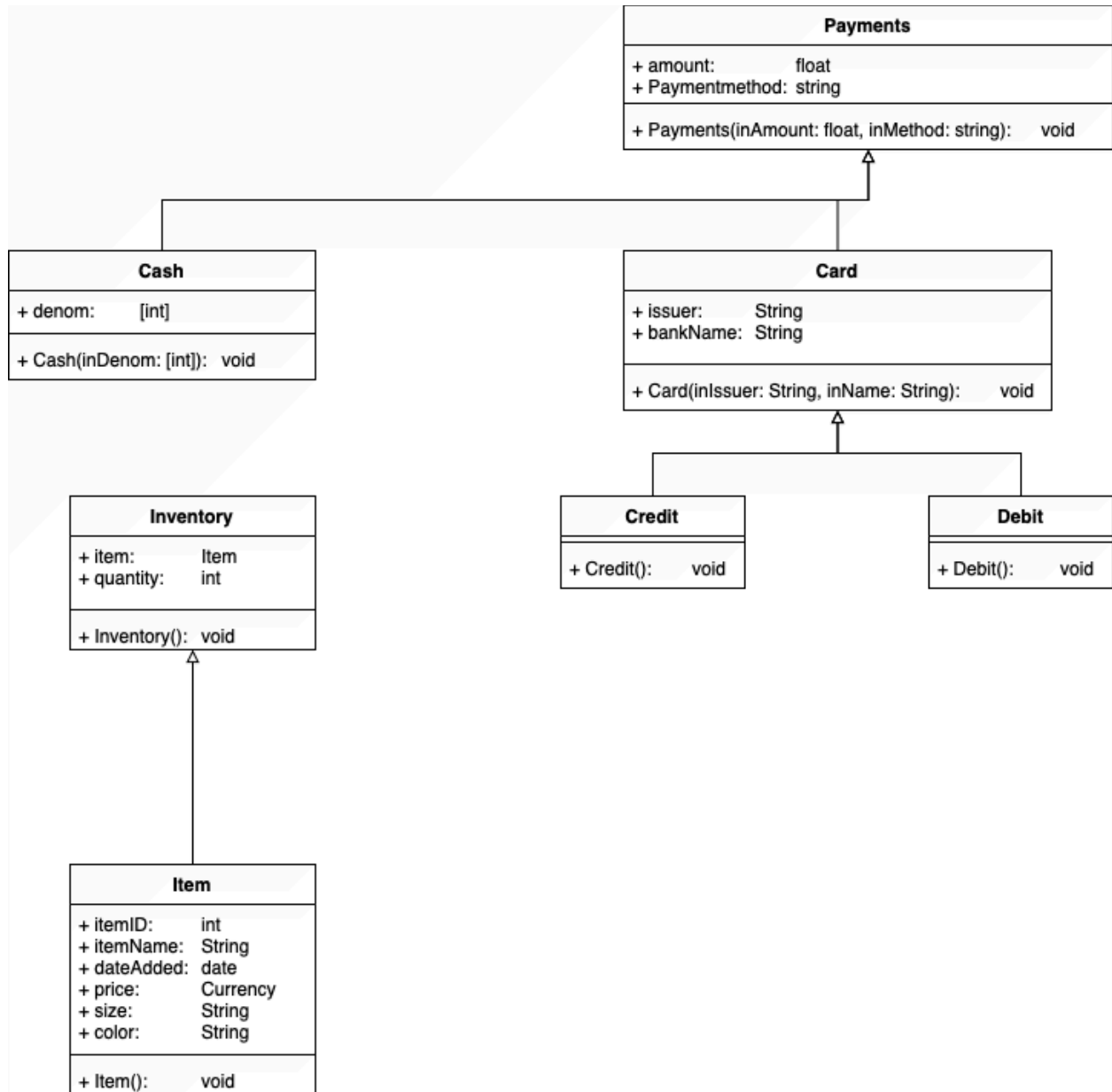
The system will be supported by both Apple and Android operating systems, and all devices will be connected to the online shared inventory. Through all these components, the overall objective of the system is to make the employees' jobs easier while simultaneously improving efficiency and organization within the stores. This, in turn, will increase sales and also provide maximum customer satisfaction.

Next, we will go over the architecture of all major components as well as the basic structure of the system and its primary functions. We will finish off with the development plan of the system and the estimated timeline according to current specifications.

Architectural Diagram



UML Diagram



Description of Classes

- Payments

- This class contains an amount as a float and a string that determines the payment method. It also has a function with two branches, with separate functions for cash and card.

a. Cash

- This class contains an array of integers that determines individual notes of cash as each element in the array. The class also has the Cash function, which is used to complete the transaction in the system.

b. Card

- This class contains two strings: the name of the bank and the name of the issuer. Depending on these two arguments, the class uses its Card function, which branches to either the Credit or Debit function.
- **Credit**
 - This function makes the transaction through the credit card company.
- **Debit**
 - This function makes the transaction through the bank connected to the debit card.

- Inventory

- This class contains the quantity as an integer and an item subclass. It also has an inventory function that updates the inventory.
- **Item** (subclass)
 - The Item subclass contains the following attributes:
 - itemID (int)
 - itemName (String)
 - dateAdded (int) → INT instead of date in Diagram?
 - price (float) → FLOAT instead of Currency in Diagram?
 - size (String)
 - color (String)
 - It also contains the Item() function.

Description of Attributes

- **Payments:**
 - a. **Amount**
 - This attribute contains a float that determines the amount to be paid by the user, either cash or card.
 - b. **Payment Method**
 - This attribute contains a string containing either “cash” or “card” to tell the system which payment the user will use.
- **Cash:**
 - a. **Denom**
 - This attribute contains an array of integers denominating the amount of currency being received (dimes, nickels, quarters, etc.).
- **Card:**
 - a. **Issuer**
 - This attribute contains a string that holds the card issuer’s information to be able to proceed with a transaction.
 - b. **Bank Name**
 - This attribute contains a string that holds the bank’s name of the card to be used to make the transaction.
- **Inventory:**
 - a. **Item**
 - This attribute contains an item’s information from the inventory to be viewed or modified from the system.
 - b. **Quantity**
 - This attribute contains an int that holds the total amount of inventory from a certain item.

- **Item:**
 - a. **Item ID**
 - This attribute contains an int that holds an item's identification number or SKU in order to be looked up manually or scanned.
 - b. **Item Name**
 - This attribute contains a string that holds the name of the item being searched.
 - c. **Date added**
 - This attribute contains a string that holds the date that the item was added or modified to the system.
 - d. **Price**
 - This attribute contains the total cost of an item
 - e. **Size**
 - This attribute contains a string array that holds the available sizes of an item.
 - f. **Color**
 - This attribute contains a string array that holds the available colors of an item.

Description of Operations

Constructors

While our UML diagram omits the getter and setter methods that are required for each class to operate, we have made sure to show constructor methods in each class and subclass. The class uses these constructor methods to initialize attribute variables with input parameters and other pre-set data.

Creating Options

One of the most significant contributors to the list of operations is the client's desire to provide options to their customers. There will be some operations used much more frequently than others, but when dealing with multiple choices of payment method, receipt handling, printing options, and others, all must be kept in mind when in the development stage; an error should not be thrown if no card is provided, for example, if the user has chosen to pay via cash.

Other Operations

Of course, the system will have many more operations than just what is listed here. Examples include methods to navigate the user interface, handling payment method APIs for digital payments, and controlling and managing the ticketing database, which provides screening information and also prevents shows from being overbooked. These are all critical features of the POS system and must be thoroughly tested to ensure seamless operation and ease of use for the client and their customers.

Delegation of Work

In total, one outline, two diagrams, and three descriptions were needed for this SDS, which were quickly divided into three sets of two tasks each. The tasks were delegated as follows:

Overview	–	Krish
Architectural Diagram	–	Andres
UML Diagram	–	Max
Classes desc.	–	Krish
Attributes desc.	–	Andres
Operations desc.	–	Max

This allowed everyone to have an equal share of the workload.